

프로그램 3.1: structure.c

설명

학생들의 성적을 관리하기 위한 구조체를 정의하고, 학생 정보를 저장하는 프로그램이다.

코드

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct studentTag {
    char name[10];
    int age;
    double gpa;
} student;

int main(void) {
    student a = {"kim", 20, 4.3};
    student b = {"park", 21, 4.2};
    return 0;
}
```

실행 결과

(출력 없음, 단순 변수 선언)

프로그램 3.2: polynomial.c

설명

다항식 덧셈을 수행하는 프로그램으로, 구조체를 이용해 다항식을 표현하고 두 개의 다항식을 더하는 기능을 구현한다.

코드

```
#include <stdio.h>
#include <stdlib.h>

#define MAX(a, b) (((a) > (b)) ? (a) : (b))
#define MAX_DEGREE 101

typedef struct {
    int degree;
```

```

    float coef[MAX_DEGREE];
} polynomial;

polynomial poly_add1(polynomial A, polynomial B) {
    polynomial C = { MAX(A.degree, B.degree), {0} };
    for (int i = 0; i <= C.degree; i++)
        C.coef[i] = A.coef[i] + B.coef[i];
    while (C.degree > 0 && C.coef[C.degree] == 0)
        C.degree--;
    return C;
}

void print_poly(polynomial p) {
    for (int i = p.degree; i >= 0; i--)
        if (p.coef[i] != 0)
            printf(i == p.degree ? "%.1fx^%d" : " + %.1fx^%d", p.coef[i], i);
    printf("\n");
}

int main() {
    polynomial a = {5, {10, 10, 10, 10, 10, 10}};
    polynomial b = {4, {1, 1, 1, 1, 1}};
    printf("A: "); print_poly(a);
    printf("B: "); print_poly(b);
    printf("-----\n");
    printf("C (A + B): "); print_poly(poly_add1(a, b));
    return 0;
}

```

실행 결과

```

A: 10.0x^5 + 10.0x^4 + 10.0x^3 + 10.0x^2 + 10.0x^1 + 10.0x^0
B: 1.0x^4 + 1.0x^3 + 1.0x^2 + 1.0x^1 + 1.0x^0
-----
C (A + B): 10.0x^5 + 11.0x^4 + 11.0x^3 + 11.0x^2 + 11.0x^1 + 11.0x^0

```

프로그램 3.3: polynomial2.c

설명

배열을 이용하여 다항식을 표현하고, 두 다항식을 더하는 기능을 구현한 프로그램이다.

코드

```

#include <stdio.h>
#include <stdlib.h>

```

```
#define MAX_TERM 101

typedef struct {
    float coef;
    int expon;
} polynomial;

polynomial terms[MAX_TERM] = { {8,3},{7,1},{1,0},{10,3},{3,2},{1,0} };
int avail = 6;

char compare(int a, int b) {
    if (a > b) return '>';
    else if (a == b) return '=';
    else return '<';
}

void attach(float coef, int expon) {
    if (avail >= MAX_TERM) {
        fprintf(stderr, "항의 개수가 너무 많음\n");
        exit(1);
    }
    terms[avail].coef = coef;
    terms[avail].expon = expon;
    avail++;
}

void poly_add2(int As, int Ae, int Bs, int Be, int *Cs, int *Ce) {
    float tempcoef;
    *Cs = avail;
    while (As <= Ae && Bs <= Be) {
        switch (compare(terms[As].expon, terms[Bs].expon)) {
            case '>':
                attach(terms[As].coef, terms[As].expon);
                As++;
                break;
            case '=':
                tempcoef = terms[As].coef + terms[Bs].coef;
                if (tempcoef) attach(tempcoef, terms[As].expon);
                As++;
                Bs++;
                break;
            case '<':
                attach(terms[Bs].coef, terms[Bs].expon);
                Bs++;
                break;
        }
    }
    for (; As <= Ae; As++)
        attach(terms[As].coef, terms[As].expon);
    for (; Bs <= Be; Bs++)
        attach(terms[Bs].coef, terms[Bs].expon);
    *Ce = avail - 1;
}
```

```

void print_poly(int s, int e) {
    for (int i = s; i < e; i++)
        printf("%.1fx^%d + ", terms[i].coef, terms[i].expon);
    printf("%.1fx^%d\n", terms[e].coef, terms[e].expon);
}

int main() {
    int As = 0, Ae = 2, Bs = 3, Be = 5, Cs, Ce;
    printf("다항식 A: ");
    print_poly(As, Ae);
    printf("다항식 B: ");
    print_poly(Bs, Be);
    poly_add2(As, Ae, Bs, Be, &Cs, &Ce);
    printf("-----\n");
    printf("다항식 C (A + B): ");
    print_poly(Cs, Ce);
    return 0;
}

```

실행 결과

```

다항식 A: 8.0x^3 + 7.0x^1 + 1.0x^0
다항식 B: 10.0x^3 + 3.0x^2 + 1.0x^0
-----
다항식 C (A + B): 18.0x^3 + 3.0x^2 + 7.0x^1 + 2.0x^0

```

프로그램 3.4: matrix.c

설명

행렬을 전치(Transpose)하는 프로그램으로, 일반 행렬과 희소 행렬을 다룬다.

코드

```

#include <stdio.h>

#define ROWS 3
#define COLS 3

void matrix_transpose(int A[ROWS][COLS], int B[ROWS][COLS]) {
    for (int r = 0; r < ROWS; r++) {
        for (int c = 0; c < COLS; c++) {
            B[c][r] = A[r][c];
        }
    }
}

```

```

}

void matrix_print(int A[ROWS][COLS]) {
    printf("-----\n");
    for (int r = 0; r < ROWS; r++) {
        for (int c = 0; c < COLS; c++) {
            printf("%d ", A[r][c]);
        }
        printf("\n");
    }
    printf("-----\n");
}

int main() {
    int array1[ROWS][COLS] = {{2,3,0},{8,9,1},{7,0,5}};
    int array2[ROWS][COLS];
    matrix_transpose(array1, array2);
    matrix_print(array1);
    matrix_print(array2);
    return 0;
}

```

실행 결과

```

-----
2 3 0
8 9 1
7 0 5
-----
-----
2 8 7
3 9 0
0 1 5
-----

```

프로그램 3.5: swap.c

설명

포인터를 사용하여 두 변수의 값을 교환하는 프로그램이다.

코드

```

#include <stdio.h>

void swap(int *px, int *py) {

```

```
    int tmp = *px;
    *px = *py;
    *py = tmp;
}

int main() {
    int a = 1, b = 2;
    printf("swap을 호출하기 전: a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("swap을 호출한 다음: a=%d, b=%d\n", a, b);
    return 0;
}
```

실행 결과

```
swap을 호출하기 전: a=1, b=2
swap을 호출한 다음: a=2, b=1
```

프로그램 3.6: malloc.c

설명

동적 메모리를 할당하여 정수 배열을 생성하고 초기화 후 출력하는 프로그램이다.

코드

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 10

int main() {
    int *p = (int *)malloc(SIZE * sizeof(int));
    if (p == NULL) {
        fprintf(stderr, "메모리가 부족해서 할당할 수 없습니다.\n");
        exit(1);
    }
    for (int i = 0; i < SIZE; i++) {
        p[i] = i;
    }
    for (int i = 0; i < SIZE; i++) {
        printf("%d ", p[i]);
    }
    printf("\n");
    free(p);
}
```

```
    return 0;
}
```

실행 결과

```
0 1 2 3 4 5 6 7 8 9
```

프로그램 3.7: malloc_struct.c

설명

동적 메모리를 사용하여 학생 구조체를 할당하고 데이터를 저장하는 프로그램이다.

코드

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char name[10];
    int age;
    double gpa;
} student;

int main() {
    student *s = (student *)malloc(sizeof(student));
    if (s == NULL) {
        fprintf(stderr, "메모리가 부족해서 할당할 수 없습니다.\n");
        exit(1);
    }
    strcpy(s->name, "Park");
    s->age = 20;
    printf("학생 이름: %s, 나이: %d\n", s->name, s->age);
    free(s);
    return 0;
}
```

실행 결과

```
학생 이름: Park, 나이: 20
```

프로그램 3.8: malloc.c

설명

동적 메모리를 할당하여 정수 배열을 생성하고 초기화 후 출력하는 프로그램이다.

코드

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 10

int main() {
    int *p = (int *)malloc(SIZE * sizeof(int));
    if (p == NULL) {
        fprintf(stderr, "메모리가 부족해서 할당할 수 없습니다.\n");
        exit(1);
    }
    for (int i = 0; i < SIZE; i++) {
        p[i] = i;
    }
    for (int i = 0; i < SIZE; i++) {
        printf("%d ", p[i]);
    }
    printf("\n");
    free(p);
    return 0;
}
```

실행 결과

```
0 1 2 3 4 5 6 7 8 9
```

프로그램 3.9: malloc_struct.c

설명

동적 메모리를 사용하여 학생 구조체를 할당하고 데이터를 저장하는 프로그램이다.

코드

```
#include <stdio.h>
#include <stdlib.h>
```



```
#include <string.h>

typedef struct {
    char name[10];
    int age;
    double gpa;
} student;

int main() {
    student *s = (student *)malloc(sizeof(student));
    if (s == NULL) {
        fprintf(stderr, "메모리가 부족해서 할당할 수 없습니다.\n");
        exit(1);
    }
    strcpy(s->name, "Park");
    s->age = 20;
    printf("학생 이름: %s, 나이: %d\n", s->name, s->age);
    free(s);
    return 0;
}
```

실행 결과

학생 이름: Park, 나이: 20
